Universidad Veracruzana
Desarrollo de Sistemas Web (DSW-19559)

P05. API REST, integración frontend/backend y persistencia

Cuarto incremento del proyecto PR09
Monitoreo de huerto o ambiente

Equipo DSW-E01
Alejandro Pacheco Luna
Juan Pablo Kuri Ricardez
Ariadna Trejo Álvarez
Pedro García Padilla

Facilitador: Dr. Gabriel Rodríguez Vásquez
Repositorio: github.com/zywoxxx/dsw-p02-monitoreo-huerto
Orizaba, Ver., 21 de octubre de 2026

Contenido

1. Qué entregamos
2. De dónde partimos y qué conservamos
3. Qué se agregó
4. El cambio de estado
5. Decisiones y alternativas
6. Modelo de datos: la migración
7. Endpoints y contratos
8. Seguridad, errores y validación
9. Despliegue y demostración
10. Pruebas y evidencia
11. Trazabilidad RF → componente
12. Problemas que tuvimos
13. Relación con la rúbrica R05
14. Limitaciones y pendientes
Anexo. Comandos

1. Qué entregamos

Este documento acompaña al repositorio del equipo y describe el cuarto incremento del PR09. En P04 diseñamos la arquitectura Web 3.0 y dejamos un esqueleto compilable con el flujo principal funcionando; cuatro operaciones del contrato quedaron solo escritas. En P05 las implementamos, completamos el cliente Angular con las vistas de alertas y anotaciones, y la API ofrece lo que pide la consigna: salud, consulta y cambio de estado. Todo corre contra la misma base PostgreSQL de los incrementos anteriores, a la que agregamos dos columnas con una migración que no borra nada.

Elemento	Dónde está
Repositorio integrado: backend Spring Boot (Java 11), frontend Angular y PostgreSQL	api-spring/, frontend-angular/, sql/, docker/
Contrato OpenAPI 0.5.0 con las 13 operaciones y catálogo legible	docs/p04/openapi.yaml, docs/p05/endpoints.md
Validaciones, manejo de errores y datos de prueba	api-spring/src/main/java/.../{alertas,anotaciones,error}, sql/02_seed.sql
Pruebas del backend (37 JUnit) y del flujo Angular-API (64 casos de contrato, 23 en navegador, 9 specs)	api-spring/src/test, scripts/pruebas_api.py, scripts/recorrido_angular.py
Evidencia del flujo extremo a extremo e índice frente a R05	docs/p05/evidencia/
README con instalación, migración, ejecución, demostración, verificación y limpieza	README.md, secciones 7 y 8
Arquitectura, modelo, trazabilidad y verificación del incremento	docs/p05/*.md
Bitácora con los fallos encontrados y cómo los corregimos	docs/bitacora.md, entradas 35 a 40

2. De dónde partimos y qué conservamos

El commit de partida es 9f398d6, el cierre de P04. Conservamos sin cambios la forma de la arquitectura (cliente Angular 16.2.12, API REST con Spring Boot 2.7.18 sobre Java 11, PostgreSQL 16), el token firmado, los tres roles con sus permisos, las reglas del huerto y el flujo principal. Lo que ya funcionaba en P04 (salud, acceso, sensores y lecturas) no se tocó: sus 30 casos del contrato siguen pasando con el mismo número.

Como en P04, el contrato se escribió antes que el código. La versión 0.5.0 agrega el cambio de estado, los detalles por identificador y pasa las anotaciones a paginadas; está validado con openapi-spec-validator y la última prueba del contrato comprueba que lo que springdoc deduce del código coincide con él.

3. Qué se agregó

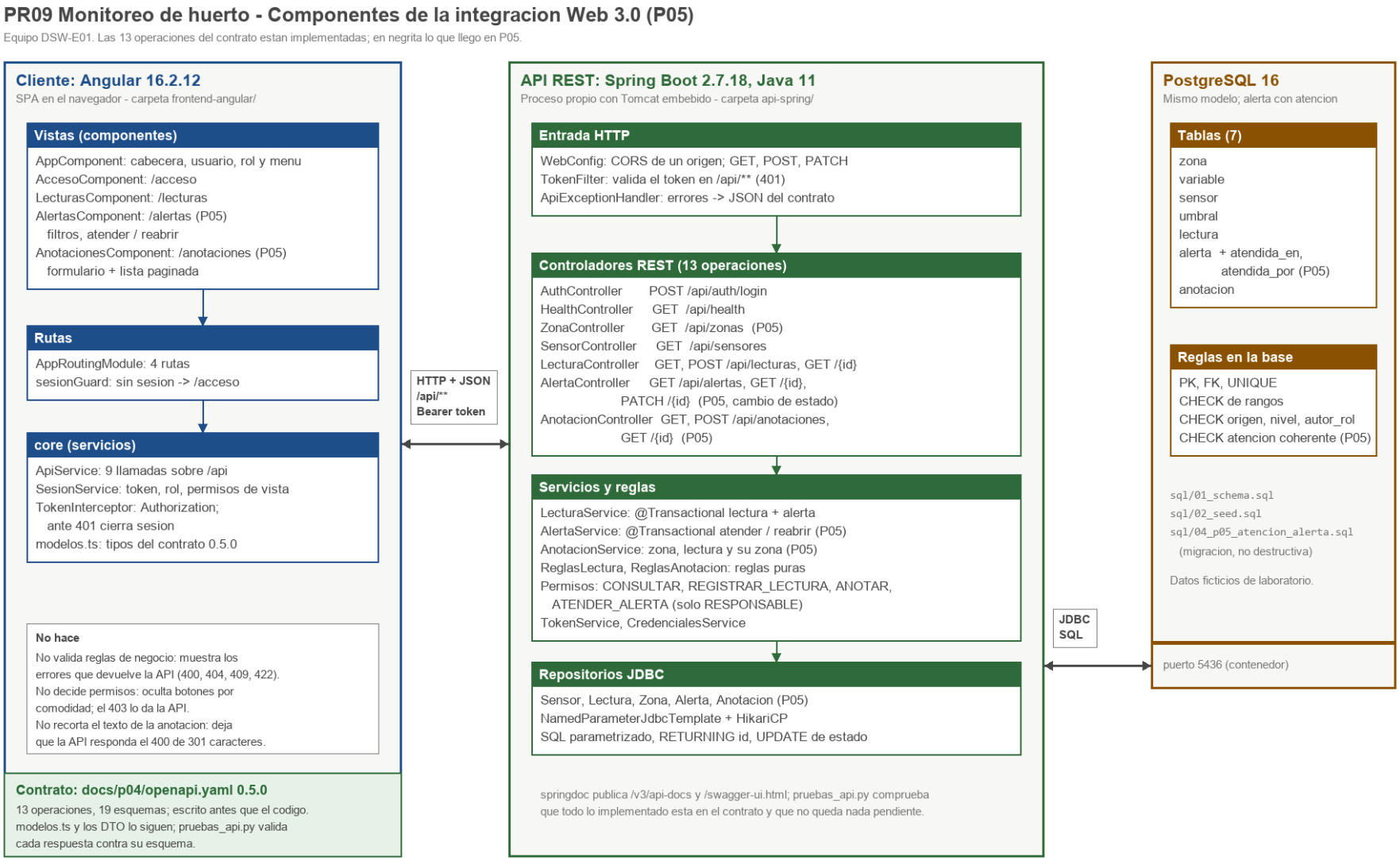


Figura 1. Componentes de la integración: las 13 operaciones implementadas; en gris lo que ya estaba en P04.

Pieza	P04	P05
Contrato	0.4.0: 10 operaciones, 4 solo definidas	0.5.0: 13 operaciones, todas implementadas
Zonas	solo contrato	GET /api/zonas
Alertas	solo contrato	GET /api/alertas con filtros y paginación, GET /{id}, PATCH /{id} (atender o reabrir)
Anotaciones	solo contrato	GET paginado, GET /{id}, POST
Permisos	CONSULTAR, REGISTRAR_LECTURA, ANOTAR	más ATENDER_ALERTA, solo para RESPONSABLE
Base	7 tablas sin cambios	mismas 7 tablas; alerta gana atendida_en y atendida_por (migración)
Cliente	acceso y lecturas	más /alertas (filtros, atender, reabrir) y /anotaciones (formulario y lista)
Pruebas contra la API en marcha	30 casos	64 casos
Recorrido en navegador	12 casos, dos roles	23 casos, los tres roles
Pruebas unitarias	16 JUnit, 5 specs	37 JUnit, 9 specs

3.1 Backend

Tres paquetes nuevos con la misma separación que lecturas: el controlador solo traduce HTTP, el servicio aplica reglas y abre la transacción, el repositorio tiene el SQL.

Paquete	Clases	Qué hace
zonas	Zona, ZonaRepository, ZonaController	GET /api/zonas ordenadas por nombre
alertas	Alerta, CambioAtencionRequest, AlertaRepository, AlertaService, AlertaController	lista con filtros nivel, atendida y zonaId; detalle; cambio de estado transaccional
anotaciones	Anotacion, AnotacionRequest, ReglasAnotacion, AnotacionRepository, AnotacionService, AnotacionController	lista paginada por zona; detalle; registro firmado con el rol del token

Permisos suma ATENDER_ALERTA y WebConfig permite el método PATCH en CORS. LecturaService, TokenFilter, ApiExceptionHandler y los repositorios de P04 son los mismos.

3.2 Frontend

Pieza	Qué hace
AlertasComponent (/alertas)	Tabla con nivel, estado y fecha; filtros por nivel y por atendida; botones "Marcar atendida" y "Reabrir" solo cuando el rol puede; para los otros roles, un aviso en lugar de los botones
AnotacionesComponent (/anotaciones)	Formulario con zona (de /api/zonas), lectura opcional y texto con contador; lista paginada con filtro por zona y el rol que firmó cada anotación

Pieza	Qué hace
ApiService	zonas(), alertas(), cambiarAtencion(), anotaciones(), registrarAnotacion()
SesionService	puedeAtenderAlertas (solo RESPONSABLE) y puedeAnotar
modelos.ts	Tipos Zona, Alerta, PaginaAlertas, CambioAtencion, NuevaAnotacion, Anotacion, PaginaAnotaciones

El cliente sigue sin decidir reglas: oculta botones por comodidad, pero el 403 lo da la API, y el texto de la anotación no se recorta en el navegador para que el 400 de 301 caracteres llegue desde el servidor y se vea.

4. El cambio de estado

El responsable marca una alerta como atendida o la reabre con PATCH /api/alertas/{id} y el cuerpo {"atendida": true | false}. El servidor registra el instante y el rol; ninguno de los dos viaja en el cuerpo. El orden de las comprobaciones es el mismo que en el registro de lecturas: identidad (401), permiso (403), forma del cuerpo (400), existencia (404), estado actual (409) y al final la escritura.

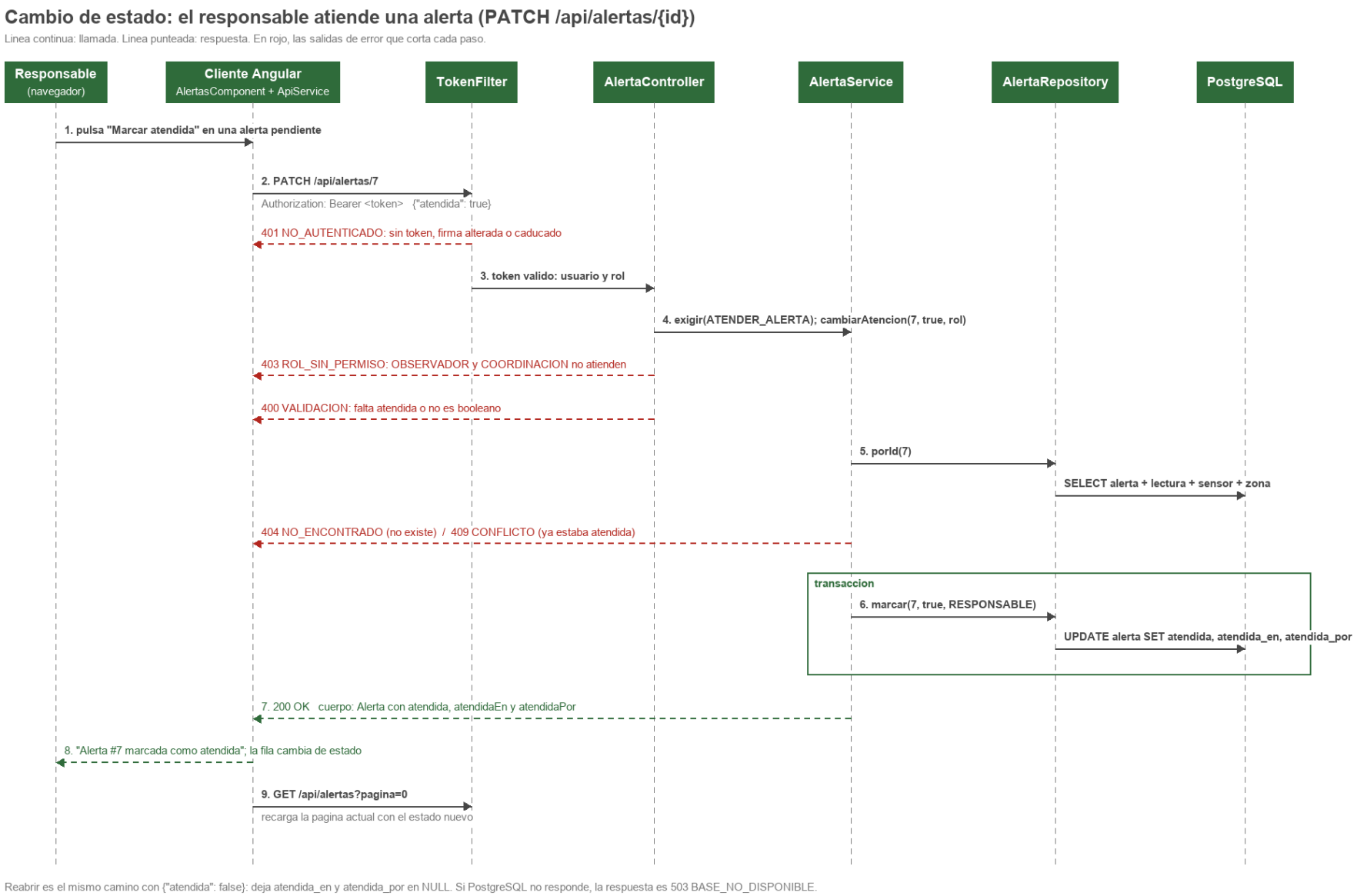


Figura 2. Secuencia de PATCH /api/alertas/{id} con sus salidas de error.

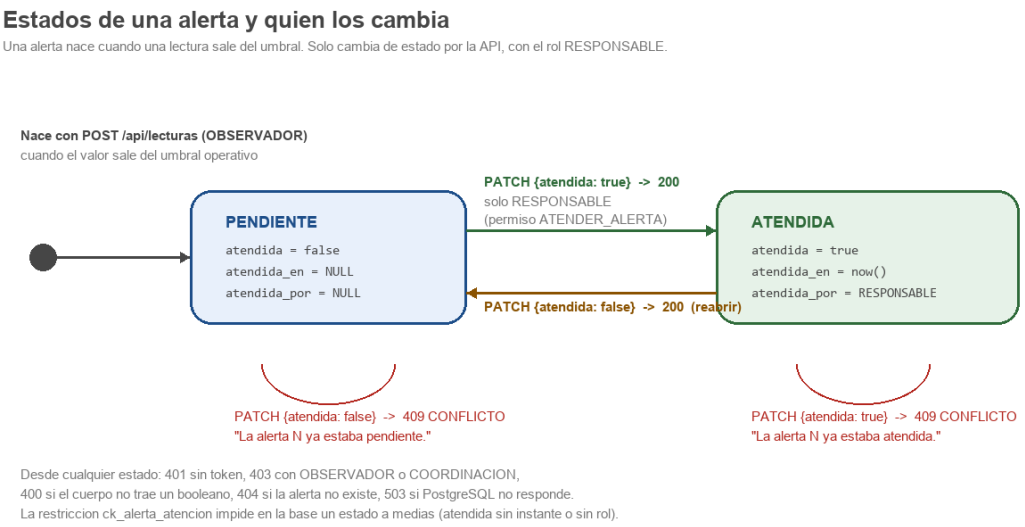


Figura 3. Estados de una alerta y quién los cambia.

5. Decisiones y alternativas

Siguen la numeración de P04 (D1 a D9). Angular y Spring Boot ya estaban decididos; estas son las decisiones propias de la integración.

#	Decisión	Alternativas que revisamos	Por qué nos quedamos con esta
D10	PATCH /api/alertas/{id} con cuerpo {atendida}	PUT con la alerta completa; POST /api/alertas/{id}/atencion como acción	Solo cambia un campo y lo demás lo decide el servidor; PUT obligaría a mandar la alerta entera y abriría la puerta a tocar campos que no deben cambiar. Reabrir es el mismo verbo con false, así que no hay dos rutas para una cosa.
D11	409 cuando el estado pedido es el actual	200 sin cambios (idempotente)	Preferimos que el cliente sepa si su acción hizo algo: un doble clic o dos personas atendiendo la misma alerta reciben "ya estaba atendida". Es el mismo uso que dimos a 409 en lecturas. Costo: el PATCH no es idempotente en sentido estricto; queda documentado.
D12	Solo RESPONSABLE atiende alertas	Responsable y coordinación; los tres roles	El acta dice que el responsable "anota decisiones" y que coordinación "consulta". Atender una alerta es decidir que ya se actuó. Es una línea en Permisos si el equipo cambia de opinión. De paso, el rol RESPONSABLE queda ejercido en la API, cosa que en P04 estaba NO_VERIFICADO.
D13	Columnas nuevas en alerta por migración no destructiva	Tabla atencion_alerta aparte; no guardar quién ni cuándo	Una tabla más sacaría el modelo del límite de siete entidades del acta. Sin instante ni rol, el cambio de estado no se puede auditar. sql/04 se puede correr dos veces y no borra nada; 01_schema.sql queda actualizado para instalaciones limpias.
D14	GET /api/anotaciones paginado (cambio 0.4 → 0.5)	Dejar la lista completa como en 0.4.0	Lecturas y alertas ya eran paginadas; una tercera forma de listar obligaría al cliente a tratar cada recurso distinto. Como el contrato va antes que el código, cambiarlo costó una versión y una nota.

#	Decisión	Alternativas que revisamos	Por qué nos quedamos con esta
D15	422 si la lectura referida no pertenece a la zona	No comprobarlo (P02 y P03 no lo hacían)	Una anotación "sobre una lectura" de otra zona es un dato incoherente que la base no puede impedir (no hay FK compuesta). Es la misma categoría que "150 °C": bien formado, pero imposible en el dominio.
D16	El cliente no recorta el texto a 300	maxlength en el textarea (lo que hacía PrimeFaces en P03)	En P03 la regla de 301 caracteres solo se pudo cubrir con JUnit porque el componente no dejaba enviarla. Aquí el contador avisa, pero el envío llega a la API y el 400 se ve en pantalla.
D17	Pruebas de integración contra la API real con Python, y MockMvc para los controladores	@SpringBootTest con Testcontainers; H2 en memoria	Testcontainers mete Docker dentro de Maven y H2 no se comporta como PostgreSQL en RETURNING, TIMESTAMPTZ y CHECK. Las pruebas Python corren contra el mismo PostgreSQL, cuentan filas antes y después y validan cada cuerpo contra el contrato.

6. Modelo de datos: la migración

El modelo sigue siendo el de P02: siete tablas. Lo único nuevo son dos columnas en alerta y una restricción que impide estados a medias.

Columna	Tipo	Desde	Para qué
atendida	BOOLEAN NOT NULL DEFAULT false	P02	estado de la alerta; hasta P04 nadie la cambiaba
atendida_en	TIMESTAMPTZ	P05	instante en que se atendió; NULL si está pendiente
atendida_por	VARCHAR(15)	P05	rol que la atendió; NULL si está pendiente

Restricción ck_alerta_atencion: una alerta atendida lleva instante y rol válidos; una pendiente no lleva ninguno. Así la base no admite una alerta "atendida" sin saber cuándo ni por quién, ni una pendiente con restos de una atención anterior.

```
ALTER TABLE alerta ADD COLUMN IF NOT EXISTS atendida_en  TIMESTAMPTZ;
ALTER TABLE alerta ADD COLUMN IF NOT EXISTS atendida_por VARCHAR(15);
-- ck_alerta_atencion solo si no existe (bloque DO ... pg_constraint):
CHECK ((atendida AND atendida_en IS NOT NULL AND atendida_por IN
('RESPONSABLE','OBSERVADOR','COORDINACION'))
       OR (NOT atendida AND atendida_en IS NULL AND atendida_por IS NULL))
```

sql/04_p05_atencion_alerta.sql se puede ejecutar varias veces y no toca filas: las alertas que ya había quedan pendientes con las columnas nuevas en NULL, que es justo el estado coherente. verify-module.sh M05 la aplica antes de arrancar la API y comprueba las columnas en information_schema (evidencia 08b_migracion_p05.txt).

Regla nueva	En la API	En PostgreSQL
Solo RESPONSABLE atiende	Permisos, AlertaController (403)	—
No repetir el estado	AlertaService (409); el UPDATE exige el estado contrario	—

Regla nueva	En la API	En PostgreSQL
Atención coherente (instante y rol juntos)	AlertaService los escribe juntos en una transacción	ck_alerta_atencion
Zona y lectura de la anotación existen	AnotacionService (404)	FK zona_id, FK lectura_id
La lectura pertenece a la zona	AnotacionService (422)	— (no hay FK compuesta)
Texto de 3 a 300 tras recortar	ReglasAnotacion (400)	CHECK length(trim(texto)) >= 3, VARCHAR(300)
Rol de la anotación	viene del token	CHECK autor_rol

7. Endpoints y contratos

El contrato formal es docs/p04/openapi.yaml, versión 0.5.0. Aquí va el resumen; los ejemplos están copiados de las respuestas reales que quedaron en la evidencia (pruebas_api.txt).

#	Método y ruta	Para qué	Quién	Éxito	Errores	Desde
1	GET /api/health	Saber si la API llega a PostgreSQL	público	200	503	P04
2	POST /api/auth/login	Cambiar credenciales por un token	público	200	400, 401	P04
3	GET /api/zonas	Zonas del huerto	3 roles	200	401	P05
4	GET /api/sensores	Catálogo con rango y umbral	3 roles	200	400, 401	P04
5	GET /api/lecturas	Historial paginado y filtrado	3 roles	200	400, 401	P04
6	GET /api/lecturas/{id}	Una lectura	3 roles	200	401, 404	P04
7	POST /api/lecturas	Registrar lectura manual	OBSERVADOR	201	400, 401, 403, 404, 409, 422, 503	P04
8	GET /api/alertas	Alertas paginadas con filtros	3 roles	200	400, 401	P05
9	GET /api/alertas/{id}	Una alerta	3 roles	200	401, 404	P05
10	PATCH /api/alertas/{id}	Cambio de estado: atender o reabrir	RESPONSABLE	200	400, 401, 403, 404, 409, 503	P05
11	GET /api/anotaciones	Anotaciones paginadas, por zona	3 roles	200	400, 401	P05
12	GET /api/anotaciones/{id}	Una anotación	3 roles	200	401, 404	P05
13	POST /api/anotaciones	Registrar anotación firmada con el rol	3 roles	201	400, 401, 404, 422, 503	P05

7.1 PATCH /api/alertas/{id}

```
> PATCH /api/alertas/48   Authorization: Bearer <token de responsable>
  {"atendida": true}
< 200 OK
```

```
  {"id":48,"lecturaId":109,"sensorCodigo":"SEN-A-TEMP-01","variable":"Temperatura
ambiente","zonaId":1,"zona":"Cama A",
   "valor":38.00,"unidad":"C","nivel":"ALTA","mensaje":"Temperatura ambiente en Cama A: 38 C por
encima del maximo 32.00 C.",
   "atendida":true,"atendidaEn":"2026-10-
21T15:58:32.434558Z","atendidaPor":"RESPONSABLE","creadaEn":"2026-10-21T15:58:19.95258Z"}

> PATCH /api/alertas/48   {"atendida": true}        (otra vez)
< 409 Conflict   {"codigo":"CONFLICTO","mensaje":"La alerta 48 ya estaba atendida.", ...}

> PATCH /api/alertas/48   {"atendida": false}       (reabrir)
< 200 OK   {"id":48, ..., "atendida":false, "creadaEn":"2026-10-21T15:58:19.95258Z"}    (sin
atendidaEn ni atendidaPor)
```

Entrada	Respuesta	Cambio en la base
{atendida: true} sobre una pendiente, token de RESPONSABLE	200 con atendida, atendidaEn y atendidaPor	atendida = true, atendida_en = now(), atendida_por = 'RESPONSABLE'
{atendida: true} otra vez	409 CONFLICTO	ninguno
{atendida: false} sobre una atendida	200 sin atendidaEn ni atendidaPor	las dos columnas vuelven a NULL
{} o {atendida: "si"}	400 VALIDACION	ninguno
id inexistente	404 NO_ENCONTRADO	ninguno
token de OBSERVADOR o COORDINACION	403 ROL_SIN_PERMISO	ninguno
sin token	401 NO_AUTENTICADO	ninguno
PostgreSQL detenido	503 BASE_NO_DISPONIBLE	ninguno

7.2 POST /api/anotaciones

Campo de entrada	Tipo	Obligatorio	Regla
zonaId	entero	sí	debe existir
lecturaId	entero	no	si viene, debe existir y pertenecer a un sensor de esa zona
texto	texto	sí	se recorta; de 3 a 300 caracteres

```
> POST /api/anotaciones   Authorization: Bearer <token de coordinacion>
  {"zonaId": 1, "texto": "[prueba] P05 anotacion de coordinacion"}
< 201 Created   Location: /api/anotaciones/42
  {"id":42,"zonaId":1,"zona":"Cama A","autorRol":"COORDINACION","texto":"[prueba] P05 anotacion de
coordinacion",
   "creadaEn":"2026-10-21T15:58:34.792856Z"}

> POST /api/anotaciones   {"zonaId": 2, "lecturaId": 108, "texto": "[prueba] P05 lectura de otra
zona"}
< 422 Unprocessable Entity   {"codigo":"REGLA_DE_NEGOCIO","mensaje":"La lectura 108 pertenece a
Cama A, no a Invernadero 1.", ...}
```

No se acepta autorRol: la anotación se firma con el rol del token. Texto "ok", de 301 caracteres o que al recortar quede en 2 → 400 con el campo señalado; zona 999 o lectura 999999 → 404; todos sin tocar la base.

8. Seguridad, errores y validación

Tema	Qué se hizo en P05
Autorización	Permiso nuevo ATENDER_ALERTA solo para RESPONSABLE; POST /api/anotaciones exige ANOTAR (los tres roles). El rol que firma la anotación y el que atiende la alerta salen del token, nunca del cuerpo
Validación	ReglasAnotacion (clase pura, con JUnit): zona obligatoria, lectura opcional positiva, texto recortado de 3 a 300. CambioAtencionRequest exige un booleano; otro tipo lo rechaza Jackson con 400
Concurrencia	El UPDATE del cambio de estado exige el estado contrario al pedido; si dos peticiones llegan a la vez solo una cambia el estado y la otra recibe 409
CORS	Se agrega PATCH a los métodos permitidos; sigue siendo un solo origen
Integridad	ck_alerta_atencion impide en la base una alerta atendida sin instante o sin rol, o pendiente con alguno de los dos
Errores	Mismo cuerpo Error en todos los casos: 400, 401, 403, 404, 409, 422 y 503. Sin trazas

Los límites de P04 siguen igual: token en sessionStorage sin revocación antes de los 30 minutos, sin HTTPS en el laboratorio, sin límite de intentos de acceso. No usamos ningún servicio externo.

9. Despliegue y demostración

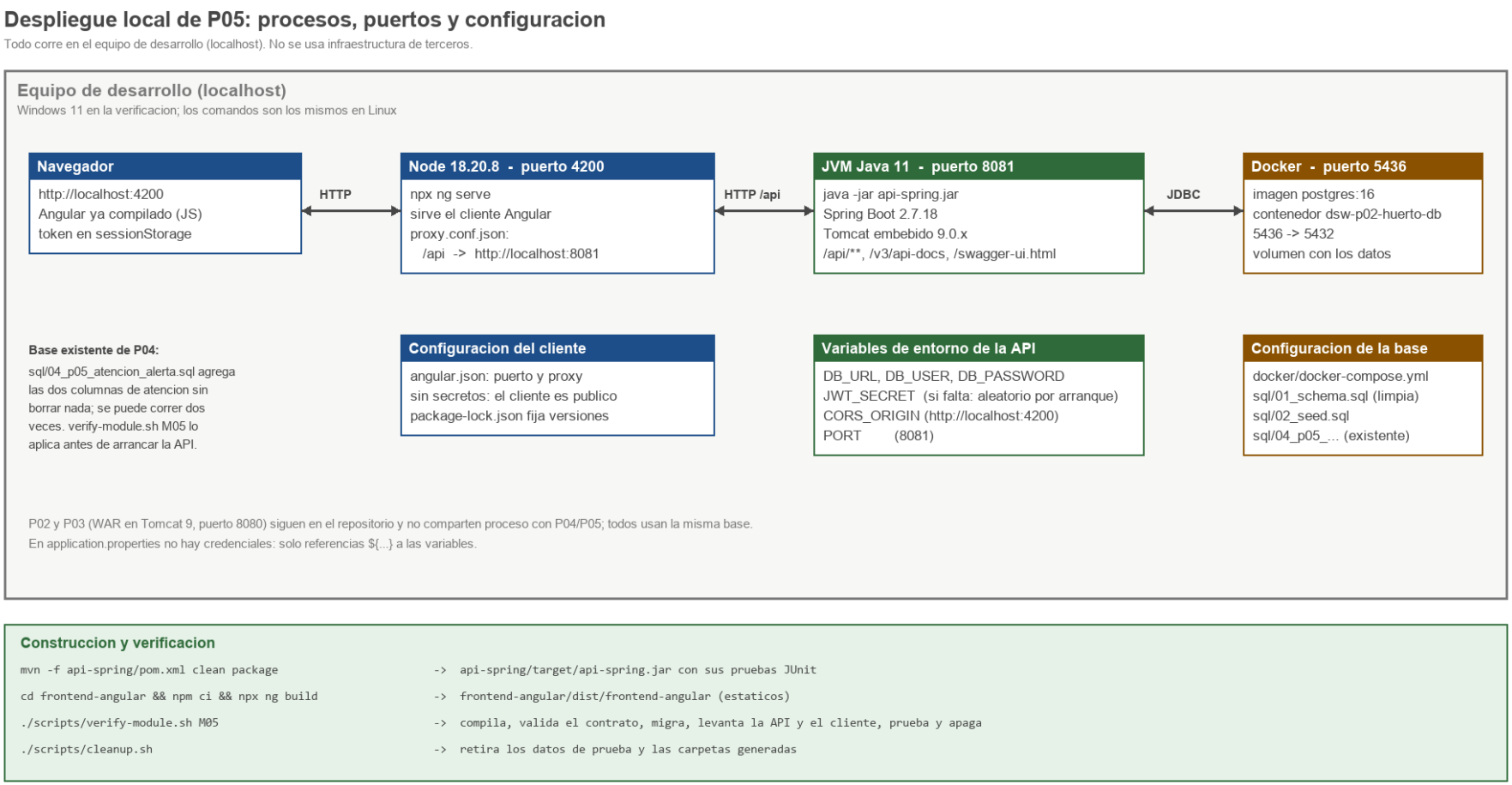


Figura 4. Despliegue local: nada cambia en procesos ni puertos; se agrega la migración.

Para mostrarlo basta un comando: ./scripts/demo_p05.sh levanta el contenedor de PostgreSQL, aplica la migración, arranca la API en el 8081 y el cliente en el 4200, espera a que respondan y abre el navegador en http://localhost:4200 y en Swagger (http://localhost:8081/swagger-ui.html). ./scripts/demo_p05.sh stop lo detiene sin tocar la base. Los usuarios de laboratorio son los mismos: observador registra lecturas, responsable atiende alertas y coordinación consulta; los tres anotan.

10. Pruebas y evidencia

Todo se regenera con ./scripts/verify-module.sh M05: compila la API y el cliente, corre sus pruebas, valida openapi.yaml, aplica la migración, arranca la API, ejecuta los 64 casos del contrato, sirve el cliente, hace el recorrido en navegador con los tres roles, consulta PostgreSQL antes y después, y apaga todo. La corrida final del 21 de octubre terminó con RESULTADO: VERIFICADO, sin fallos.

```
verify-module.sh M05 - 2026-10-21T09:59:16-06:00 - commit 701016e
VERIFICADO    | Java 11 disponible
VERIFICADO    | Node 18 disponible
VERIFICADO    | api-spring compila con Java 11 y sus pruebas pasan
VERIFICADO    | El jar contiene controladores (lecturas/LecturaController zonas/ZonaController
alertas/AlertaController anotaciones/AnotacionController), driver de PostgreSQL, Spring JDBC y
springdoc
VERIFICADO    | application.properties toma conexion y secreto de variables de entorno
VERIFICADO    | frontend-angular compila y genera dist/frontend-angular
VERIFICADO    | Las pruebas del cliente Angular pasan
VERIFICADO    | openapi.yaml es OpenAPI 3.0 valido
VERIFICADO    | PostgreSQL responde y el modelo de 7 tablas existe
VERIFICADO    | La migracion de P05 se aplica sin error y alerta tiene atendida_en y atendida_por
VERIFICADO    | La API arranca y /api/health responde 200
VERIFICADO    | /api/health reporta db=UP
VERIFICADO    | pruebas_api.py termino sin casos NO_VERIFICADO
VERIFICADO    | recorrido_angular.py termino sin casos NO_VERIFICADO
VERIFICADO    | Las lecturas y anotaciones creadas por la API existen en PostgreSQL
VERIFICADO    | La API se detuvo y el puerto 8081 quedo libre
Fallos: 0
```

10.1 Contrato contra la API en marcha (64 casos)

Cada caso valida el cuerpo contra el esquema del contrato y, en los negativos, cuenta filas en PostgreSQL antes y después. Los 30 primeros son los de P04 sin cambios; del 30 al 63 son de P05 y el 64 compara el OpenAPI generado con el escrito.

```
01 | VERIFICADO    | Salud publica: 200, base UP
02 | VERIFICADO    | Login correcto: 200 con token y rol OBSERVADOR
03 | VERIFICADO    | Login incorrecto: 401 sin token
04 | VERIFICADO    | Login incompleto: 400
05 | VERIFICADO    | Recurso protegido sin token: 401
06 | VERIFICADO    | Token con firma alterada: 401
07 | VERIFICADO    | Catalogo de sensores: 200 con rango fisico y umbral
08 | VERIFICADO    | Parametro de consulta no numerico: 400
09 | VERIFICADO    | Historial paginado: 200 y total igual al de PostgreSQL
10 | VERIFICADO    | Filtro por procedencia: solo lecturas simuladas
11 | VERIFICADO    | Procedencia desconocida: 400
12 | VERIFICADO    | Lectura valida: 201, Location, procedencia manual, sin alerta, +1 lectura
13 | VERIFICADO    | La lectura creada se recupera por su identificador
14 | VERIFICADO    | Lectura sobre el umbral: 201 con alerta ALTA, +1 lectura y +1 alerta
```

```
15 | VERIFICADO    | Lectura bajo el umbral: 201 con alerta BAJA
16 | VERIFICADO    | Limites inclusivos: 15 y 32 se guardan sin alerta
17 | VERIFICADO    | Rechazo: valor no numerico "abc" -> 400 VALIDACION, la base no cambia
18 | VERIFICADO    | Rechazo: valor con tres decimales 24.555 -> 400 VALIDACION, la base no cambia
19 | VERIFICADO    | Rechazo: sin valor -> 400 VALIDACION, la base no cambia
20 | VERIFICADO    | Rechazo: sin sensor -> 400 VALIDACION, la base no cambia
21 | VERIFICADO    | Rechazo: JSON mal formado -> 400 VALIDACION, la base no cambia
22 | VERIFICADO    | Rechazo: observacion de 201 caracteres -> 400 VALIDACION, la base no cambia
23 | VERIFICADO    | Rechazo: valor 150, fuera del rango fisico -> 422 REGLA_DE_NEGOCIO, la base no cambia
24 | VERIFICADO    | Rechazo: sensor 999, no existe -> 404 NO_ENCONTRADO, la base no cambia
25 | VERIFICADO    | Sensor inactivo: 409 y la base no cambia
26 | VERIFICADO    | Registrar sin token: 401 y la base no cambia
27 | VERIFICADO    | Rol sin permiso de captura: 403 al registrar, 200 al consultar
28 | VERIFICADO    | Lectura inexistente: 404
29 | VERIFICADO    | CORS: solo se admite el origen del cliente Angular
30 | VERIFICADO    | Catalogo de zonas sin token: 401
31 | VERIFICADO    | Catalogo de zonas: 200 con las dos zonas ordenadas por nombre
32 | VERIFICADO    | Alertas paginadas: 200 y total igual al de PostgreSQL
33 | VERIFICADO    | Filtro por nivel: solo alertas ALTA
34 | VERIFICADO    | Nivel desconocido: 400
35 | VERIFICADO    | Filtro por estado: solo alertas pendientes
36 | VERIFICADO    | Parametro booleano no valido: 400
37 | VERIFICADO    | La alerta creada por la lectura 38 se recupera pendiente
38 | VERIFICADO    | Alerta inexistente: 404
39 | VERIFICADO    | OBSERVADOR no atiende alertas: 403 y la base no cambia
40 | VERIFICADO    | COORDINACION no atiende alertas: 403 y la base no cambia
41 | VERIFICADO    | RESPONSABLE atiende la alerta: 200, rol e instante los pone el servidor, PostgreSQL
actualizado
42 | VERIFICADO    | Repetir el mismo estado: 409 CONFLICTO y la base no cambia
43 | VERIFICADO    | Sin el campo atendida: 400 VALIDACION senalando el campo, la base no cambia
44 | VERIFICADO    | atendida no booleano: 400 y la base no cambia
45 | VERIFICADO    | Atender una alerta inexistente: 404
46 | VERIFICADO    | Cambiar el estado sin token: 401 y la base no cambia
47 | VERIFICADO    | Reabrir la alerta: 200 sin atendidaEn ni atendidaPor; ambas columnas NULL en PostgreSQL
48 | VERIFICADO    | Anotaciones paginadas: 200 y total igual al de PostgreSQL
49 | VERIFICADO    | Parametro de consulta no numerico: 400
50 | VERIFICADO    | Anotacion de zona: 201, Location, firmada con el rol del token, +1 anotacion
51 | VERIFICADO    | Anotacion sobre una lectura de la zona: 201 con lecturaId y rol OBSERVADOR
52 | VERIFICADO    | La anotacion creada se recupera por su identificador
53 | VERIFICADO    | Anotacion inexistente: 404
54 | VERIFICADO    | Rechazo: texto "ok" (2 caracteres) -> 400 VALIDACION, la base no cambia
55 | VERIFICADO    | Rechazo: texto de 301 caracteres -> 400 VALIDACION, la base no cambia
56 | VERIFICADO    | Rechazo: texto "   ab   " (recortado quedan 2) -> 400 VALIDACION, la base no cambia
57 | VERIFICADO    | Rechazo: sin zonaId -> 400 VALIDACION, la base no cambia
58 | VERIFICADO    | Rechazo: sin texto -> 400 VALIDACION, la base no cambia
59 | VERIFICADO    | Rechazo: lecturaId -3 -> 400 VALIDACION, la base no cambia
60 | VERIFICADO    | Rechazo: zonaId 999, no existe -> 404 NO_ENCONTRADO, la base no cambia
61 | VERIFICADO    | Rechazo: lecturaId 999999, no existe -> 404 NO_ENCONTRADO, la base no cambia
62 | VERIFICADO    | Rechazo: zonaId 2 con la lectura 108, que es de la zona 1 -> 422 REGLA_DE_NEGOCIO, la base
no cambia
63 | VERIFICADO    | Anotar sin token: 401 y la base no cambia
64 | VERIFICADO    | Todo lo implementado esta en el contrato y no hay operaciones del contrato sin implementar
```

10.2 Recorrido del cliente en navegador (23 casos)

```
01 | VERIFICADO    | Sin sesion, /lecturas lleva a la pantalla de acceso
02 | VERIFICADO    | Acceso con contrasena incorrecta: mensaje de error y sin sesion
03 | VERIFICADO    | Acceso del observador: formulario de captura e historial desde la API
04 | VERIFICADO    | Lectura valida 25.5: 201, mensaje, fila nueva y +1 en PostgreSQL
05 | VERIFICADO    | Lectura 38 fuera del umbral: 201 con alerta ALTA visible y +1 alerta
06 | VERIFICADO    | Valor no numerico: 400, mensaje por campo, se conserva lo escrito, la base no
cambia
07 | VERIFICADO    | Valor 150 fuera del rango fisico: 422 y la base no cambia
```

```
08 | VERIFICADO    | Filtro por procedencia: el historial solo muestra lecturas simuladas
09 | VERIFICADO    | Salir borra la sesion del navegador
10 | VERIFICADO    | Rol COORDINACION: consulta el historial pero no ve el formulario de captura
11 | VERIFICADO    | Rol RESPONSABLE en /alertas: la alerta ALTA del recorrido aparece pendiente con
boton de atender
12 | VERIFICADO    | Marcar atendida: PATCH 200, mensaje, etiqueta con el rol y atendida = true en
PostgreSQL
13 | VERIFICADO    | Reabrir: PATCH 200 y la alerta vuelve a pendiente en pantalla y en PostgreSQL
14 | VERIFICADO    | Anotacion con texto "ok": 400 VALIDACION en pantalla, lo capturado se conserva, la
base no cambia
15 | VERIFICADO    | Anotacion de 301 caracteres: el cliente no recorta (sin maxlength) y la API
responde 400
16 | VERIFICADO    | Lectura relacionada 999999: 404 NO_ENCONTRADO en pantalla y lo capturado se
conserva
17 | VERIFICADO    | Anotacion valida sin lectura: 201, mensaje, fila nueva firmada por RESPONSABLE y +1
en PostgreSQL
18 | VERIFICADO    | Filtro por zona Invernadero 1: la anotacion de Cama A no aparece
19 | VERIFICADO    | Rol COORDINACION en /alertas: consulta sin botones de atender y con el aviso del
rol
20 | VERIFICADO    | Rol COORDINACION anota: 201 y la fila queda firmada con COORDINACION
21 | VERIFICADO    | Rol OBSERVADOR en /alertas: consulta sin botones de atender
22 | VERIFICADO    | Token alterado en el navegador: la API responde 401 y el cliente vuelve al acceso
23 | VERIFICADO    | Documentacion generada por springdoc disponible en /swagger-ui.html
RESUMEN: 23/23 casos VERIFICADO
```

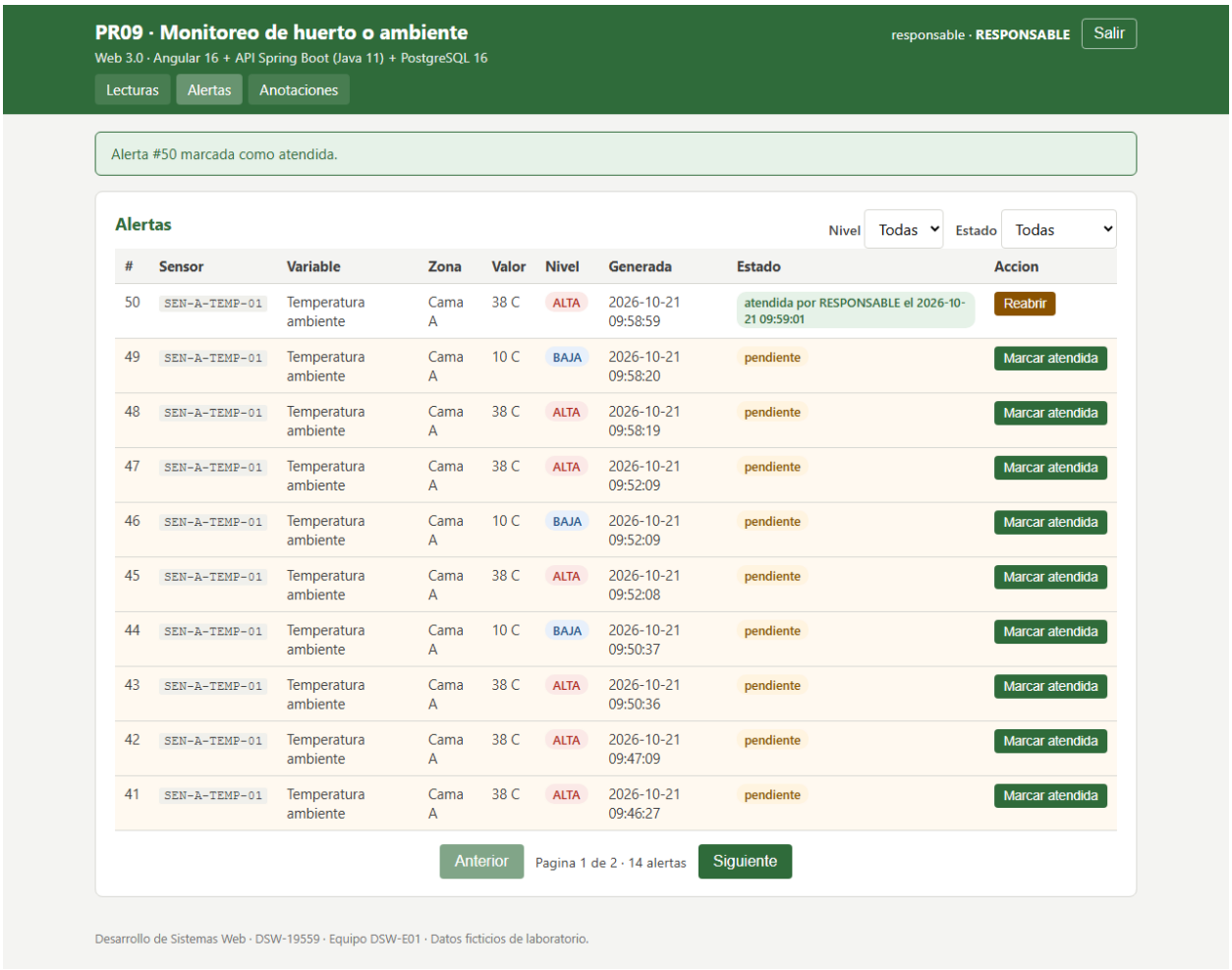


Figura 5. El responsable marca la alerta como atendida: etiqueta con rol y fecha; la fila deja de estar pendiente.

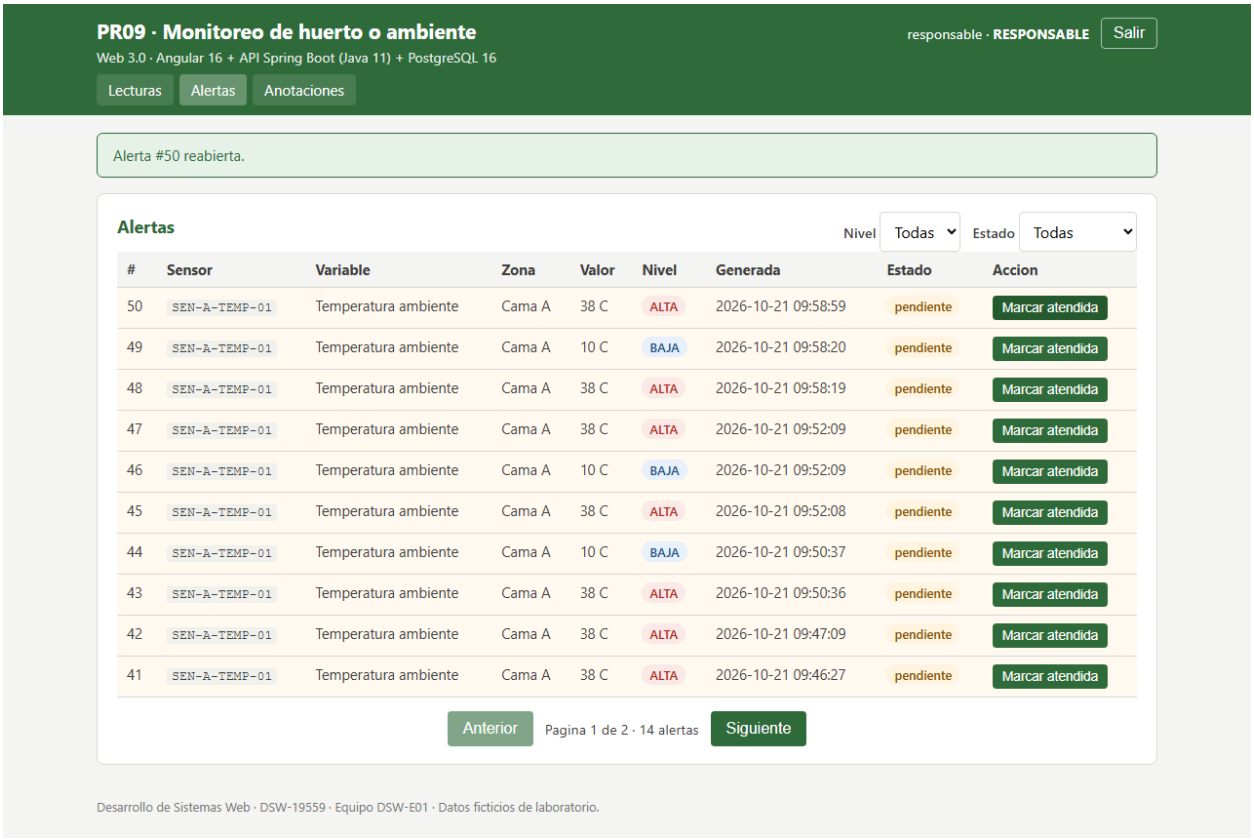


Figura 6. La misma alerta reabierta: vuelve a pendiente y el botón cambia.

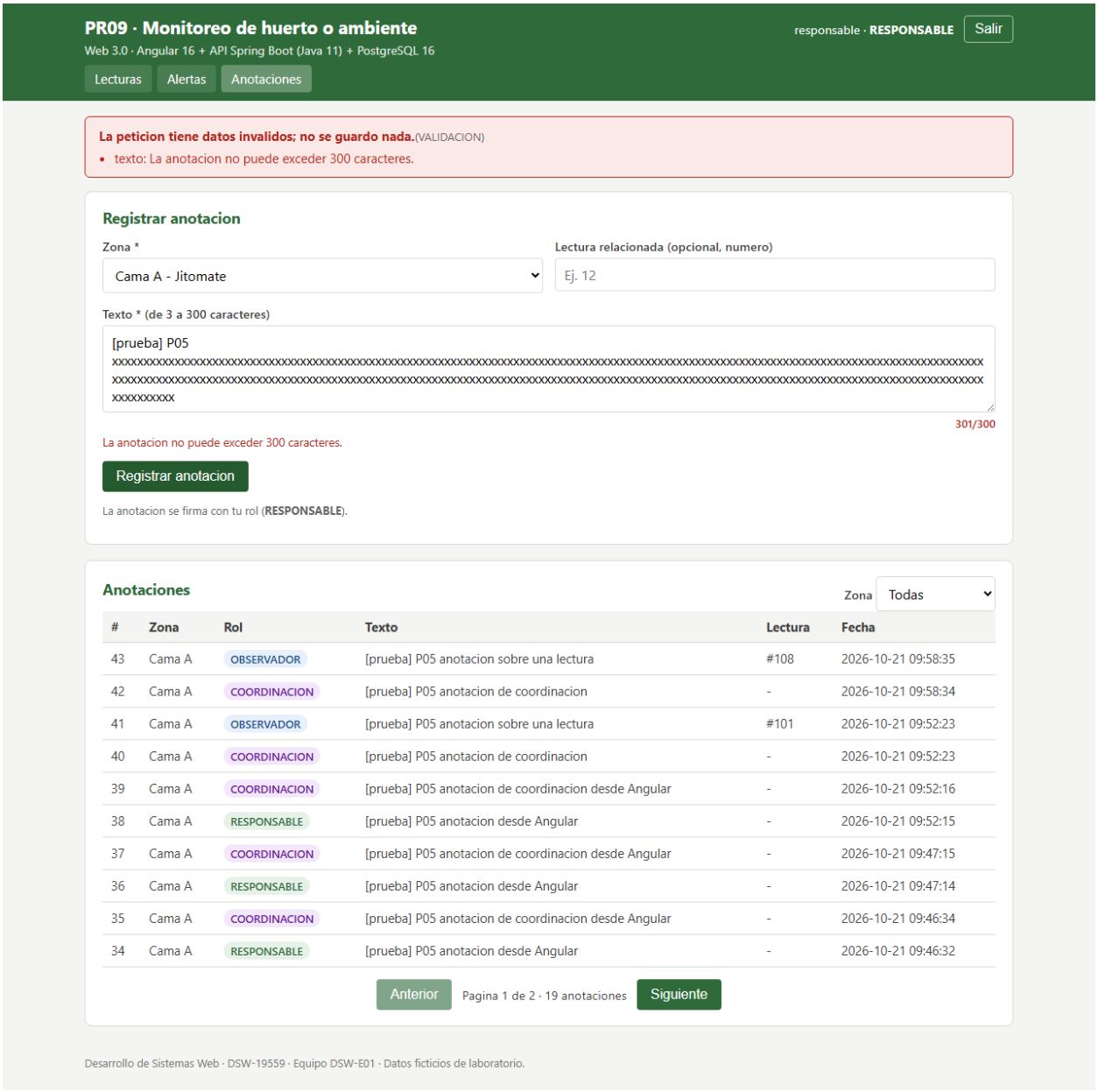


Figura 7. 301 caracteres: el cliente no recorta y muestra el 400 de la API con el campo señalado.

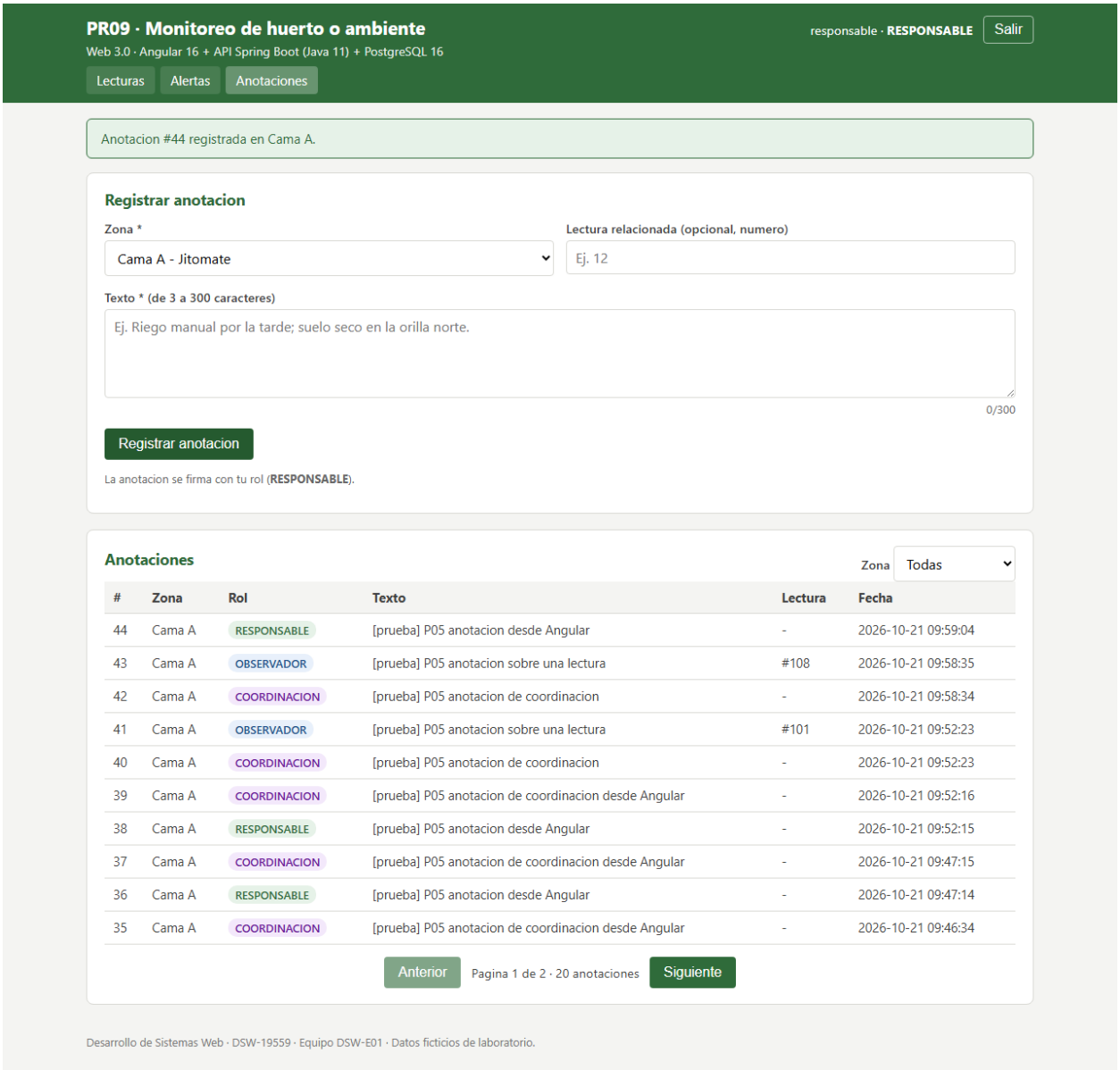


Figura 8. Anotación registrada y firmada con el rol RESPONSABLE; aparece primera en la lista.

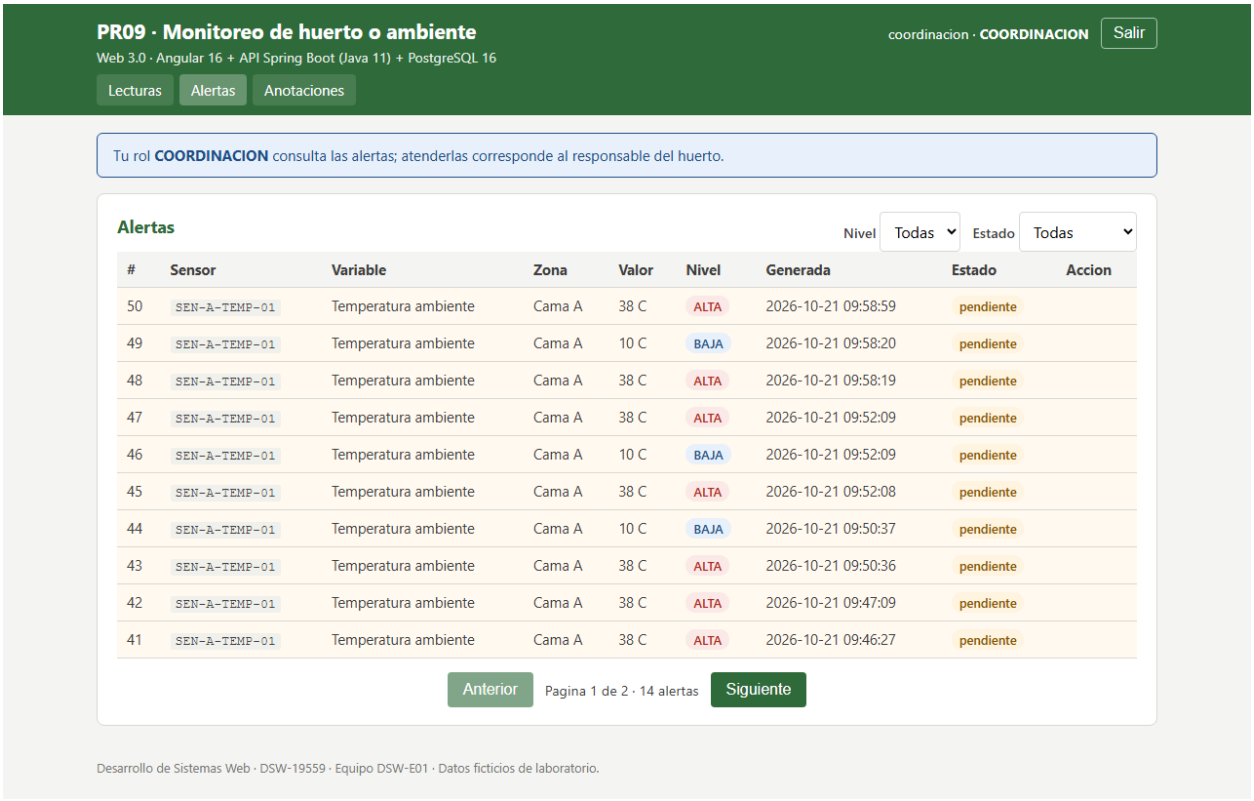


Figura 9. Coordinación consulta las alertas sin botones de atender y con el aviso del rol.



PR09 Monitoreo de huerto - API REST

0.5.0 OAS3

/v3/api-docs

Generado desde el codigo. El contrato de referencia es docs/p04/openapi.yaml.

Servers

http://localhost:8081 - Generated server url

lectura-controller

- GET /api/lecturas
- POST /api/lecturas
- GET /api/lecturas/{id}

auth-controller

- POST /api/auth/login

anotacion-controller

- GET /api/anotaciones
- POST /api/anotaciones
- GET /api/anotaciones/{id}

alerta-controller

- GET /api/alertas/{id}
- PATCH /api/alertas/{id}
- GET /api/alertas

zona-controller

- GET /api/zonas

sensor-controller

- GET /api/sensores

health-controller

- GET /api/health

Schemas

- LecturaRequest
- AlertaResumen
- Lectura
- Credenciales
- AnotacionRequest
- Anotacion
- CambioAtencionRequest
- Alerta

Figura 10. Las 13 operaciones que springdoc deduce del código, en /swagger-ui.html.

10.3 Base de datos detenida

Con docker stop sobre el contenedor (sin borrar nada), la salud, la consulta, el registro de lectura, el cambio de estado y la anotación responden 503; el login sigue funcionando porque no usa la base; al volver el contenedor la API se recupera sin reiniciarla.

```
--- GET /api/health
[estado 200, 0.008036 s]
--- login de un usuario que no existe
[estado 401, 0.055645 s]
--- login de observador con contrasena incorrecta
[estado 401, 0.046275 s]
--- GET /api/health
[estado 503, 5.019948 s]
--- GET /api/sensores con token
[estado 503, 5.030393 s]
--- POST /api/lecturas con token
[estado 503, 5.016539 s]
--- PATCH /api/alertas/1 con token de responsable
[estado 503, 5.022551 s]
--- POST /api/anotaciones con token
[estado 503, 5.023206 s]
--- POST /api/auth/login (no usa la base)
[estado 200, 0.045458 s]
--- GET /api/health (la API se recupera sin reiniciarla)
[estado 200, 0.006629 s]
```

11. Trazabilidad RF → componente

RF	Requisito	Operación	Componentes (cliente / API / base)	Prueba	Estado
RF01	Zonas	GET /api/zonas	select de zona y filtro / ZonaController, ZonaRepository / zona	contrato 30, 31; navegador 17, 18	VERIFICADO
RF03	Registrar lecturas	POST /api/lecturas	LecturasComponent / LecturaService, ReglasLectura / lectura	contrato 12, 17 a 26; navegador 04, 06, 07	VERIFICADO (de P04)
RF05	Historial	GET /api/lecturas	tabla y paginador / LecturaService / lectura	contrato 09 a 13, 28; navegador 08	VERIFICADO (de P04)
RF06a	Alerta automática y su consulta	objeto alerta en POST; GET /api/alertas, /{id}	AlertasComponent / AlertaController, AlertaService, AlertaRepository / alerta	contrato 14, 15, 32 a 38; navegador 05, 11, 19, 21	VERIFICADO
RF06b	Cambio de estado de la alerta	PATCH /api/alertas/{id}	botones atender/reabrir / AlertaService.cambiarAtencion (@Transactional) / atendida, atendida_en, atendida_por, ck_alerta_atencion	contrato 39 a 47; navegador 12, 13, 19, 21; JUnit	VERIFICADO
RF06c	Anotaciones por rol	GET, POST /api/anotaciones, /{id}	AnotacionesComponent / AnotacionService, ReglasAnotacion / anotacion	contrato 48 a 63; navegador 14 a 18, 20; JUnit	VERIFICADO

RF	Requisito	Operación	Componentes (cliente / API / base)	Prueba	Estado
RF07	Modelo relacionado	todas	— / repositorios / 7 tablas, migración sql/04	03_consultas_verificacion.sql; paso de migración	VERIFICADO
RF08	API Web 3.0	las 13 del contrato	ApiService / api-spring / —	contrato 64	VERIFICADO: 13 de 13
RNF01	Ruta de salud	GET /api/health	— / HealthController / —	contrato 01; base detenida	VERIFICADO (200 y 503)
RNF02	Autorización por rol	todas	sesionGuard, SesionService / TokenFilter, Permisos / —	contrato 05, 06, 26, 27, 39, 40, 46, 63; navegador 19, 21, 22	VERIFICADO: los 3 roles

La matriz completa, con la lectura inversa (de componente a requisito) y la continuidad P02 → P05, está en docs/p05/trazabilidad-r05.md.

12. Problemas que tuvimos

Qué pasó	Cómo apareció	Qué hicimos
La primera corrida del verificador terminó con 2 NO_VERIFICADO: el caso 16 del contrato vio una alerta de más y el caso 02 del navegador recibió un 504 del proxy	verify-module.sh M05, primera corrida	Una ejecución anterior que detuvimos a medias dejó vivo su recorrido en navegador; corrió en paralelo contra la misma API y metió lecturas entre los conteos. No era defecto de la aplicación: se detuvieron los procesos sobrantes y la corrida limpia dio 0 fallos. Queda en la bitácora porque la guía pide conservar el primer error
Condición de carrera al atender una alerta: el servicio leía el estado y hacía el UPDATE sin condición, así que dos peticiones simultáneas podían atender la misma alerta y ambas recibir 200	revisión de código	El UPDATE exige ahora atendida = NOT :atendida; si no cambia ninguna fila, el servicio responde 409 igual que si el estado ya fuera el pedido
El contrato 0.4.0 definía GET /api/anotaciones como lista completa, distinto de lecturas y alertas	al escribir el cliente	Se cambió a paginado en 0.5.0 (decisión D14) y se anotó en el contrato

Los tres están en docs/bitacora.md (entradas 35 a 40) con hora, comando y resultado tal cual salió.

13. Relación con la rúbrica R05

Criterio	Pts	Dónde está
API Spring funcional: endpoints y respuestas verificables	3	Sección 7 y 10.1; pruebas_api.txt con la petición y la respuesta real de los 64 casos; openapi_generado.json; figura de Swagger
Integración Angular: consumo de servicios y flujo de usuario	3	Sección 3.2 y 10.2; recorrido_angular.txt (23 casos, tres roles) y capturas 01 a 22; 9 specs

Criterio	Pts	Dónde está
Persistencia PostgreSQL: operaciones y estado reproducible	2	Sección 6; 08_postgres_antes.txt, 08b_migracion_p05.txt, 12_postgres_despues.txt; casos 41 y 47 comprueban las columnas en la base
Errores y validación: casos inválidos y respuestas controladas	2	Secciones 7 y 8; casos 17 a 27, 34, 36, 38 a 46, 49, 53 a 63; 14_base_detenida.txt; capturas 06, 07, 14, 15, 16
Pruebas y repositorio: resultados, README y trazabilidad	2	Sección 10 y 11; 01_build_api.txt (37 JUnit), 06_pruebas_frontend.txt, 13_resumen.txt; README secciones 7 y 8; docs/p05/trazabilidad-r05.md; bitácora

El índice que relaciona cada archivo de evidencia con su criterio está en docs/p05/evidencia/INDICE.md.

14. Limitaciones y pendientes

Asunto	Estado	Nota
Dos PATCH simultáneos sobre la misma alerta	NO_VERIFICADO	La condición del UPDATE lo cubre por construcción; no hay prueba de concurrencia
Caducidad real del token a los 30 minutos	NO_VERIFICADO	Probada con reloj fijo en JUnit
Petición CORS real desde otro origen en el navegador	NO_VERIFICADO	Se prueba el preflight con peticiones directas
Servir el resultado de ng build; despliegue en Linux	NO_VERIFICADO	Solo ng serve en Windows 11
Reproducción por otro integrante en máquina limpia	PENDIENTE	CA05
Confirmación del facilitador en F02	PENDIENTE	No se afirma que esté confirmada

Otras limitaciones: la atención de una alerta guarda el rol, no la persona (no hay tabla de usuarios, por el acta); un token emitido no se puede revocar; los usuarios están en un archivo dentro del jar; springdoc nombra los esquemas según las clases Java; los datos de prueba quedan en la base hasta correr cleanup.sh.

Anexo. Comandos

```
docker compose -f docker/docker-compose.yml up -d        # la misma base de P02 a P04
docker exec -i dsw-p02-huerto-db psql -U huerto_app -d huerto_db -v ON_ERROR_STOP=1 <
sql/04_p05_atencion_alerta.sql
cd frontend-angular && npm ci && cd ..                   # una vez

# Demostracion (API 8081 + cliente 4200 + navegador)
./scripts/demo_p05.sh            # ./scripts/demo_p05.sh stop para detener

# Verificacion y limpieza (con la API y el cliente detenidos)
./scripts/verify-module.sh M05
./scripts/prueba_base_detenida.sh
./scripts/cleanup.sh
```